MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

SMART HOME AUTOMATION SYSTEM

A Mini Project Report submitted to
Microcontrollers and Embedded Systems Lab (ECE 3143)

Submitted by

Registration Number	Roll No	Name
240959002	02	Nyasha Singh
240959008	05	Shivansh Bose
240959024	10	Garv Kishore Gupta
240959040	11	Ashwini Kodethur

SCHOOL OF ELECTRICAL AND ELECTRONICS ENGINEERING
MANIPAL-576104, KARNATAKA, INDIA

NOVEMBER 2026

ABSTRACT

Residential automation is moving from isolated gadgets towards integrated systems that handle entry control, ventilation, lighting and safety alerts together, while keeping the occupant informed from a distance. Low-cost ARM-based microcontrollers now carry the peripherals needed to host all of these functions on a single board, which makes such systems attractive both as products and as teaching platforms. The objective of this project is to design and demonstrate a miniature smart-home prototype in which one STM32F411CEU6 microcontroller integrates access control, environmental monitoring, automatic lighting, fan control, hazard alerting and wireless status reporting.

The prototype is built on a hand-made model house. A 4×3 keypad and a servo form a password-protected main door, while an RC522 RFID reader and a second servo operate a garage with timed closing. A DHT22 sensor sets the speed of a PWM-driven fan through temperature and humidity thresholds, an MQ-2 sensor read through the 12-bit ADC drives a buzzer when a configured count is reached, and a PIR sensor with an LDR module switches a room LED only for motion in darkness. An OLED shows the keypad status and a compact sensor line. The firmware is written in C with the STM32 HAL as a non-blocking, timestamp-driven loop. An HC-05 module on USART2 sends a text status report about every two seconds to a Windows computer, from which a PowerShell bridge and a Java server feed a web dashboard and ThingSpeak, and supported text commands allow remote control of the devices.

Functional tests of the integrated model show that the password 1234 releases the main door for 3 s, the authorised RFID tag opens the garage for 10 s without disturbing the keypad screen, the fan moves between four states as the configured temperature and humidity limits are crossed, the buzzer responds when the MQ-2 reading reaches 350 ADC counts, the room LED lights only for motion in darkness, and telemetry and command acknowledgements are exchanged over Bluetooth. These results show that independent subsystems can share one processor, one display and one serial link without blocking each other.

It is concluded that the STM32F411CEU6 offers sufficient GPIO, timer, ADC, I²C, SPI and USART resources to run the complete design at low cost. The prototype is educational: the password and RFID authentication are fixed and weak, the MQ-2 threshold is an uncalibrated raw count, and power use and response times were not measured. Stronger authentication, calibrated sensing, authenticated remote commands and direct Wi-Fi connectivity are the natural next steps.

Tools and hardware: STM32F411CEU6 (Black Pill), SH1106-compatible OLED, 4×3 keypad, RC522, DHT22, MQ-2, PIR, LDR module, HC-05, two micro servos, DC fan, buzzer and LEDs; embedded C with STM32 HAL and STM32CubeMX, a PowerShell serial bridge, a Java server, the ThingSpeak platform and an HTML/JavaScript dashboard.

Index Terms—*Embedded systems, smart home, access control, STM32F411, Bluetooth telemetry, environmental sensing, PWM fan control, web dashboard.*

Contents

		Page No
	Abstract	**ii**
Chapter 1	**INTRODUCTION**	**1**
	1.1 Introduction	1
	1.2 Objectives	1
	1.3 Organization of Report	2
Chapter 2	**METHODOLOGY**	**2**
	2.1 Introduction	2
	2.2 Methodology	2
	2.3 Detailed Methodology	2
	2.4 Circuit Layout / Block Diagrams	3
	2.5 Component Specifications	4
	2.6 Justification for Component Selection	4
	2.7 Tools Used	4
	2.8 Listing of Components, Devices, Software and Data Sheets	4
	2.9 Preliminary Result Analysis	5
	2.10 Conclusions	5
Chapter 3	**RESULT ANALYSIS**	**5**
	3.1 Introduction	5
	3.2 Photograph of the Working Project	5
	3.3 Result Analysis	6
	3.4 Graphical / Tabular Form	6
	3.5 Explanation of the Graphical / Tabulated Results	7
	3.6 Significance of the Results Obtained	7
	3.7 Deviations from Expected Results and Justification	7
	3.8 Conclusions	7
Chapter 4	**CONCLUSION AND FUTURE SCOPE OF WORK**	**7**
	4.1 Work Conclusion	7
	4.2 Future Scope of Work	8
	REFERENCES	**8**
	ANNEXURES (OPTIONAL)	**8**
	Plagiarism Report	**11**
	Project Details	**Last page**

CHAPTER 1
INTRODUCTION

1.1 Introduction

This chapter sets out the context of the work. It describes the present-day scenario of home automation, reviews published work on Bluetooth-, Wi-Fi- and Internet-of-Things-based home control, and places this prototype against that background. It then states the objectives, target specifications and importance of the outcome, and closes with the organisation of the report.

Present-day scenario. Homes contain a growing number of electrically controlled functions, such as door locks, lighting, ventilation and alarms, that were traditionally operated separately and by hand. Cheap microcontrollers, short-range wireless modules and cloud services now make it practical to sense and control these functions together [1]. For the occupant the gains are convenience (lights and fans that react to presence and climate), safety (early warning of smoke or gas), security (controlled entry) and the ability to see the state of the house from a computer. For an engineering student a house is also a convenient system for practising embedded design, since it combines digital inputs, analogue sensing, timed actuation, serial communication and a user interface in one product.

Literature review. Early research concentrated on replacing wired switches with a wireless link. Sriskanthan *et al.* [2] described a Bluetooth-based controller for household appliances, and Piyare and Tazil [3] later switched appliances from a mobile phone over Bluetooth through a microcontroller board, showing that a low-cost serial Bluetooth module is adequate for short-range control. Wi-Fi was explored by ElShafee and Hamed [4], who connected a home controller to a network so that appliances could be reached from ordinary network devices. As the Internet of Things matured, work moved towards remote monitoring and cloud connectivity: Piyare [5] reported a smartphone-based home control and monitoring system, Pavithra and Balakrishnan [6] an IoT-based monitoring and control system, and Kodali *et al.* [7] a design that combines security functions with appliance automation. Gubbi *et al.* [1] describe the wider architecture in which sensing nodes, communication links and cloud services form separate layers.

In these studies the emphasis is usually on one aspect, most often appliance switching or remote access. This project aims at a broader, laboratory-scale combination on a single ARM microcontroller: local access control by keypad and RFID, climate-dependent fan speed, a gas-triggered alert, presence-and-darkness lighting, a local display, and a wireless telemetry and command link feeding a web dashboard. The design uses only the on-chip peripherals of the STM32F411CEU6 and shows how the subsystems share the processor, the display and the serial link without blocking one another.

1.2 Objectives

Main objective. To build and demonstrate a smart-home prototype on an ARM-family microcontroller that integrates access control, environmental monitoring, automatic lighting, ventilation control, alerts and wireless status reporting.

Secondary objectives. The secondary objectives are listed below.

- Protect the main door with a keypad password and a servo.
- Provide RFID-based garage access with timed closing.
- Measure temperature and humidity with a DHT22 and control fan speed by PWM according to configured thresholds.
- Read the MQ-2 analogue output and sound a buzzer at a configured threshold.
- Switch a room LED according to PIR motion and LDR darkness.
- Show password status and compact sensor readings on an OLED.
- Transmit telemetry over an HC-05 Bluetooth link to a Windows PC and accept supported remote commands.
- Display live values and device states on a web dashboard and retain history through the server and ThingSpeak.

Target specifications. The specifications the prototype is designed to meet are summarised in Table 1.1.

TABLE 1.1
TARGET SPECIFICATIONS OF THE PROTOTYPE

Parameter	Target
Controller	STM32F411CEU6 (Arm Cortex-M4), 16 MHz HSI clock
Main door	4-digit password; servo unlocks for 3 s, then relocks
Garage	Authorised RFID UID opens servo; closes after 10 s
DHT22 sampling	About every 2 s
Fan (PWM)	OFF: T < 25 °C or RH < 80 %; LOW: T ≥ 25 °C and RH ≥ 80 %; MEDIUM: T ≥ 28 °C and RH ≥ 85 %; HIGH: T ≥ 31 °C and RH ≥ 90 %
MQ-2 alert	Sampled about every 500 ms; buzzer alert at ≥ 350 ADC counts
Room light	30 s PIR stabilisation; LED on only for motion in darkness; 10 s window
Telemetry	Text report about every 2 s over HC-05 (USART2)
Display	OLED, I²C address 0x3C, keypad status and sensor line

Importance of the end result. The prototype demonstrates, in a small and inspectable form, how one low-cost microcontroller can coordinate several unrelated subsystems. It gives a reusable pin plan, a simple text command protocol and a data path to a dashboard that later projects can extend. It also records honestly where the design is only a classroom

demonstration, so that its limits are not mistaken for those of a security- or safety-rated product.

1.3 Organization of Report

Chapter 1 introduces the subject, reviews the literature and states the objectives and target specifications. Chapter 2 describes the methodology: the working of each subsystem, the block diagram and pin plan, component specifications and selection, the tools used and a preliminary analysis. Chapter 3 presents the result analysis, including photographs of the working model, tabulated and graphical results, their significance and the deviations from expectation. Chapter 4 summarises the work, states the conclusions and outlines the future scope. The references, annexures, the plagiarism report and the project details follow.

CHAPTER 2
METHODOLOGY

2.1 Introduction

This chapter explains how the system was designed and built. It describes the overall approach, the working of every subsystem, the block diagram, pin allocation and firmware flow, the specification of the components and the reasons for choosing them, and the software tools used. It ends with a design-level analysis made before the results are presented.

2.2 Methodology

The system is organised as a set of independent subsystems that meet only at three shared resources: the main loop and its system tick, the OLED, and the Bluetooth link. Sensors and inputs are read by the STM32F411CEU6 [8], which applies the configured rules and drives the servos, the fan, the buzzer and the lights. Timed behaviour (door relock, garage close, light window, sensor sampling) is handled with timestamps against the system tick instead of blocking delays, so that no subsystem stops the others. A physical model house with separate rooms carries the sensors and actuators, and the wiring is made on a breadboard with jumper wires.

Data leave the board through an HC-05 Bluetooth module. A Windows computer receives them on a Bluetooth COM port, a PowerShell bridge forwards the messages to a Java server, the server keeps the latest values for the website and periodically uploads readings to ThingSpeak [9] for history. The firmware is written in C on the STM32 HAL [10].

2.3 Detailed Methodology

Keypad, OLED and main-door servo. The 4×3 keypad is scanned by driving the four row lines (PB0–PB3) and reading the three column lines (PA15, PB12, PB13), which have pull-ups. The entered digits are masked on the OLED; # submits and * clears the entry. The buffer holds up to eight characters while the valid password is four digits (1234). A match shows ACCESS GRANTED and turns the main-door servo (TIM1_CH1, PA8) to the unlocked position for 3 s before relocking; a mismatch shows ACCESS DENIED. The top line of the OLED continuously shows compact temperature, humidity and raw MQ-2 values, and RFID handling never redraws the keypad screen. The display is on I²C1 (PB6 SCL, PB7 SDA) at address 0x3C [11].

RFID and garage servo. The RC522 reader [12] uses SPI1 (PA5 SCK, PA6 MISO, PA7 MOSI) with chip-select on PA4 and reset on PB10. It reads the unique identifier (UID) of ISO/IEC 14443A tags [13]. One authorised UID is stored in the firmware; a second known tag is configured as unauthorised. The authorised tag turns the garage servo (TIM1_CH2, PA9) open, and a non-blocking timer closes it after 10 s. UID-only authentication is acceptable for a classroom prototype but not for a production lock.

DHT22 and fan. The DHT22 [14] on PB14 is sampled about every 2 s, and temperature and humidity are stored in tenths of a unit. The fan is driven by PWM on TIM3_CH1 (PB4). In automatic mode the fan is OFF if $T < 25.0$ °C or $RH < 80.0$ %; LOW if both limits are met but the higher limits are not; MEDIUM if $T \geq 28.0$ °C and $RH \geq 85.0$ %; and HIGH if $T \geq 31.0$ °C and $RH \geq 90.0$ %. The PWM compare values are 0, 350, 650 and 1000. If a read fails, the previous fan setting is kept. Bluetooth commands can force the fan on (HIGH), off, or return it to automatic mode.

MQ-2 and buzzer. The analogue output of the MQ-2 [15] is connected to PA0 (ADC1_IN0) and read about every 500 ms. The raw count is shown on the OLED and in the telemetry. At or above the threshold of 350 counts (*MQ2_THRESHOLD*) the buzzer (PB9) alert is activated; below it, the buzzer follows a manual state if one has been set and is otherwise off. The threshold is experimental and does not correspond to a calibrated concentration.

PIR, LDR and room LED. The PIR output (PA1) and the LDR module output D0 (PB15) are read as digital inputs; the firmware takes PIR high as motion and LDR D0 high as darkness. After a 30 s start-up stabilisation, the room LED (PB8) is switched on only when motion occurs in darkness. It stays on for a 10 s window; a PIR re-check at about 9 s restarts the window if motion continues, otherwise the LED turns off. Brightness alone never turns the LED on, and Bluetooth can force the light on, off or back to automatic.

HC-05 Bluetooth link. The HC-05 [16] is connected to USART2: PA2 (TX) to the module RXD and PA3 (RX) from its TXD. The firmware receives line-oriented, case-sensitive commands (80-byte line buffer) and sends a readable multi-line status report about every 2 s. The fields are temperature, humidity, smoke ADC value and status, motion, light level, room light, fan mode and speed, main door, garage door, garage light, DHT22 status and RFID reader status.

Commands such as DOOR_OPEN, GARAGE_OPEN, FAN_AUTO or ROOMLIGHT_ON are acknowledged with ACK strings, and unknown commands return ERR,UNKNOWN_COMMAND (Annexure A). A smoke alarm can override a manual buzzer-off request.

Windows bridge, server and dashboard. The module appears on Windows as a Bluetooth COM port. A PowerShell bridge forwards the received reports to a Java server, which parses and stores the latest values and sends periodic readings to ThingSpeak. The website shows live values from the server and history from ThingSpeak. The dashboard design has overview cards for temperature, humidity, MQ-2 count and motion, temperature/humidity and MQ-2 trend charts, device cards and controls, an event history, a sensor page with CSV export, and a settings page with a demo mode whose values are illustrative only. Keys and tokens are kept out of the report.

2.4 Circuit Layout / Block Diagrams

Fig. 2.1 shows the block diagram. Six inputs feed the controller and six output groups are driven from it, while a bidirectional serial path through the HC-05 connects the board to the computer, the server, ThingSpeak and the dashboard. The pin allocation is listed in Table 2.1 and the firmware flow in Fig. 2.2.

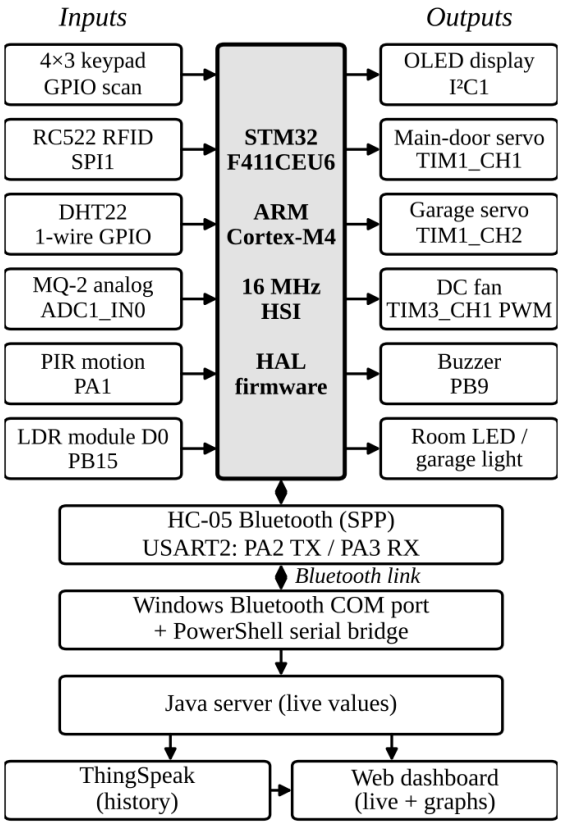


Fig. 2.1. Block diagram of smart home automation system.

TABLE 2.1
STM32F411CEU6 PIN ALLOCATION

Function	Pin / peripheral
OLED SCL / SDA (0x3C)	PB6 / PB7 (I²C1)
Keypad rows R1–R4	PB0, PB1, PB2, PB3 (outputs)
Keypad columns C1–C3	PA15, PB12, PB13 (pull-up inputs)
Main-door servo	PA8 (TIM1_CH1)
Garage servo	PA9 (TIM1_CH2)
MQ-2 analogue output	PA0 (ADC1_IN0)
PIR output	PA1
LDR module D0	PB15
Room LED / buzzer	PB8 / PB9
DHT22 data	PB5
Fan PWM	PB4 (TIM3_CH1)
RC522 CS / SCK / MISO / MOSI	PA4 / PA5 / PA6 / PA7 (SPI1)
RC522 reset	PB10
HC-05 RXD ← / TXD →	PA2 (USART2_TX) / PA3 (USART2_RX)

The pin plan comes from the integrated firmware and must be checked against the final wiring before it is treated as a schematic. The UART is crossed (STM32 TX to HC-05 RXD), all grounds are common, and the 3.3 V logic of the STM32 must never receive a 5 V signal. PA15 is shared with the JTAG interface, so the debug configuration must leave it free for the keypad.

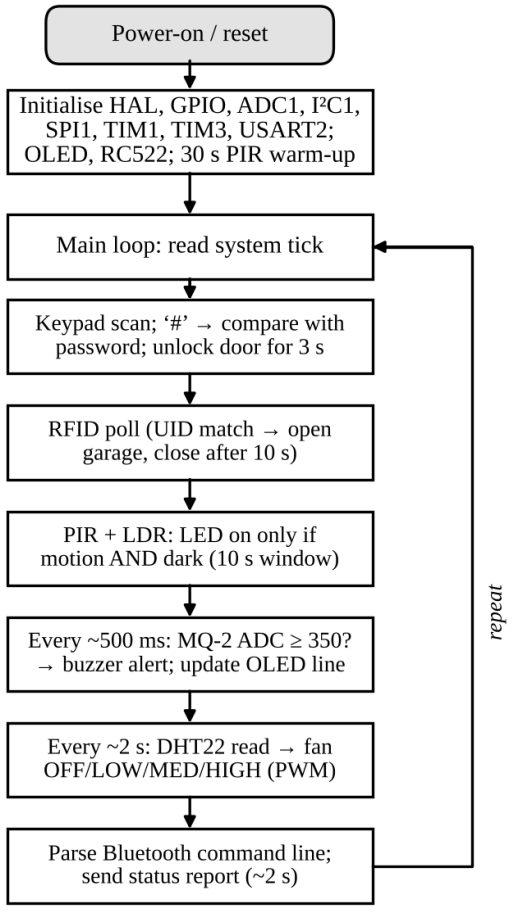


Fig. 2.2. Control flow of the integrated firmware.

2.5 Component Specifications

Table 2.2 lists the main components with their relevant characteristics. Values are taken from the cited data sheets and must be confirmed against the exact modules fitted.

TABLE 2.2
COMPONENT SPECIFICATIONS

Component	Key characteristics	Role
STM32F411CEU 6 (Black Pill)	Arm Cortex-M4F up to 100 MHz, 512 KB flash, 128 KB SRAM, 12-bit ADC, timers, I²C, SPI, USART [8]	Main controller
OLED, SH1106-compatible	128×64 pixels, I²C, address 0x3C	Local interface
4×3 matrix keypad	12 keys, 7 lines (4 rows, 3 columns)	Password entry
Micro servos (2)	PWM-positioned hobby servo	Main door, garage
RC522 RFID reader	13.56 MHz, ISO/IEC 14443A, SPI, 3.3 V [12]	Garage access
DHT22 (AM2302)	Digital output, −40 to 80 °C, 0–100 % RH, ≥ 2 s between reads [14]	Temperature , humidity
MQ-2 module	SnO_2 sensing layer with heater; analogue output; needs warm-up [15]	Smoke / gas level
PIR sensor	Digital output, high on motion	Motion
LDR module	Light-dependent resistor with comparator; digital D0	Dark / bright
DC fan, buzzer, LEDs	Fan by PWM through a driver stage; buzzer and LEDs by GPIO	Ventilation, alerts, lights
HC-05	Bluetooth SPP, 2.4 GHz, 3.3 V logic, 9600 baud 8-N-1 in data mode [16]	Wireless serial link

2.6 Justification for Component Selection

- **STM32F411CEU6:** an ARM device with enough GPIO, three kinds of serial interface, a 12-bit ADC and advanced and general timers for PWM, so no external multiplexer or second controller is needed; it also meets the course requirement for an ARM-family processor.
- **DHT22:** a single-wire digital sensor that gives temperature and humidity with finer resolution and better accuracy than the DHT11, at low cost.
- **MQ-2:** inexpensive, responds to smoke and several combustible gases, and gives an analogue output suited to the ADC.
- **RC522:** low-cost SPI reader for common ISO/IEC 14443A cards and tags.
- **OLED:** needs only two I²C lines and shows both the keypad prompt and sensor values.
- **HC-05:** acts as a wireless serial cable and appears as a COM port on Windows without extra drivers.
- **Servos:** a small servo is a convenient model of a door or garage lock.
- **ThingSpeak:** a ready-made IoT platform that stores time-stamped readings and draws charts, though it limits the update rate (commonly about 15 s on free accounts) [9].

2.7 Tools Used

Firmware is developed in C on top of the STM32 HAL [10], with peripheral initialisation of the CubeMX style (for example *MX_USART2_UART_Init()*) and a 16 MHz HSI clock with the PLL disabled. On the computer side, the Windows Bluetooth COM port, a PowerShell serial bridge, a Java server and the ThingSpeak service carry the data, and the dashboard is a single-file HTML, CSS and JavaScript application. A Windows terminal was used to observe the telemetry and to send commands.

2.8 Listing of Components, Devices, Software and Data Sheets

TABLE 2.3
LISTING OF HARDWARE, SOFTWARE AND REFERENCE DOCUMENTS

Category	Items
Hardware	STM32F411CEU6 Black Pill; SH1106-compatible OLED; 4×3 keypad; RC522 with tags; DHT22; MQ-2 module; PIR sensor; LDR module; two micro servos; DC fan; buzzer; room LED and garage light; HC-05; breadboard, jumper wires and a handmade model house
Software	Embedded C, STM32 HAL, CubeMX-style initialisation; PowerShell bridge; Java server; ThingSpeak; HTML/JavaScript dashboard; Windows Bluetooth COM port
Test aids	OLED display; Windows serial terminal for telemetry and commands; browser for the dashboard
Data sheets	STM32F411 datasheet and reference manual [8], [17]; HAL manual [10]; MFRC522 [12]; AM2302 [14]; MQ-2 [15]; HC-05 [16]; I²C specification [11]

2.9 Preliminary Result Analysis

Before the functional tests, the design was checked arithmetically. These figures follow from the configuration and are not measurements.

- **Pin budget.** The design uses 26 GPIO lines: 7 for the keypad, 2 for the OLED, 2 servo PWM, 1 for MQ-2, 1 each for PIR, LDR, room LED, buzzer, DHT22, fan PWM and garage light, 5 for the RC522 and 2 for the HC-05.
- **MQ-2 threshold.** The 12-bit ADC has 4095 counts at full scale. With a 3.3 V reference, 350 counts corresponds to about $350/4095 \times 3.3\ \text{V} \approx 0.28\ \text{V}$ at PA0.
- **Fan PWM.** The duty cycle is the compare value divided by the timer period. If the period is 1000 counts, the settings 350, 650 and 1000 give 35 %, 65 % and 100 %; the actual period must be read from the timer configuration.
- **Serial load.** At 9600 baud with 10 bits per byte the link carries about 960 bytes/s. A report of roughly 300 bytes (estimated) every 2 s would use about 15 % of this capacity.
- **Command buffer.** The longest command, GARAGE_LIGHT_OFF, has 16 characters, well inside the 80-byte line buffer.
- **Sampling.** The 2 s DHT22 interval matches the minimum interval between reads required by its data sheet [14].

2.10 Conclusions

The methodology divides the house into independent subsystems that share only the main loop, the OLED and the Bluetooth link. A single STM32F411CEU6 provides all required interfaces, timed behaviour is handled without blocking, and the data path to the dashboard is separated into a live path and a history path. The preliminary analysis shows that the pin budget, ADC threshold, PWM settings and

serial load are consistent with the hardware, and the next chapter reports how the assembled model behaved.

CHAPTER 3
RESULT ANALYSIS

3.1 Introduction

This chapter presents the results obtained from the assembled model. It begins with photographs of the working project, then reports the functional tests in tabular and graphical form, explains them, discusses their significance, and records the deviations from the expected behaviour together with their justification.

3.2 Photograph of the Working Project

Fig. 3.1 to Fig. 3.4 show the working model from different angles. The model is a handmade blue-and-white house with separate rooms, with the sensors and actuators fixed to the walls and floor and the controller and breadboard wiring placed in one room.

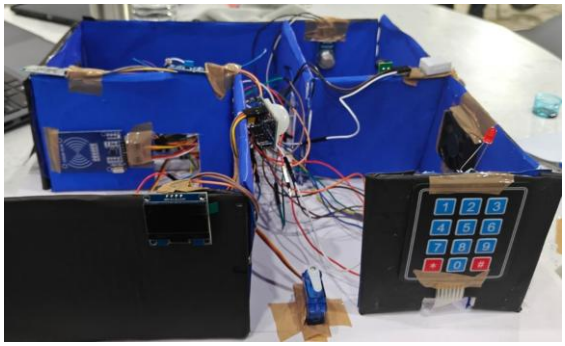


Fig. 3.1. Front view of the working prototype: RC522 RFID reader on the left wall, OLED and 4×3 keypad on the front panels, a servo in the foreground, and PIR sensor, MQ-2, DHT22, fan and LED in the rooms behind.

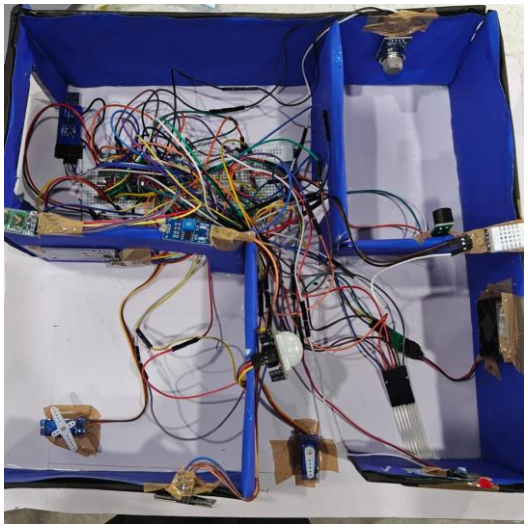


Fig. 3.2. Top view of the complete model showing the room layout, the breadboard with the controller and the wiring, and the sensors, servos, buzzer, fan and LED.

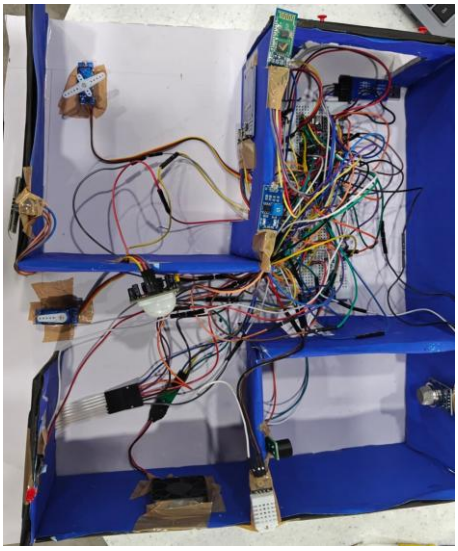


Fig. 3.3. Top view of the model from the HC-05 side, showing the HC-05 module, LDR module, PIR sensor, two servos, fan and DHT22.

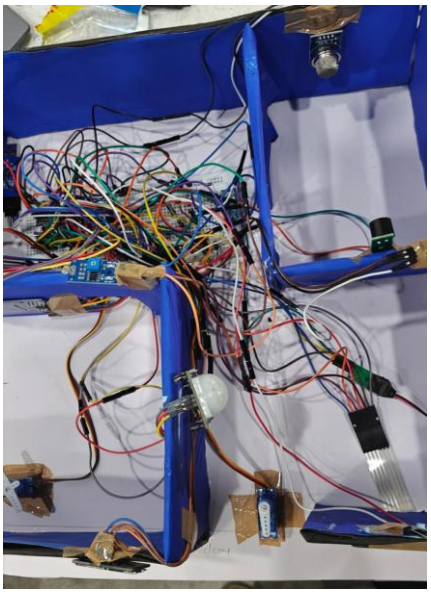


Fig. 3.4. Close view of the wiring and sensor placement: breadboard and jumper wires, LDR module, PIR sensor, MQ-2, buzzer, DHT22 and keypad ribbon.

3.3 Result Analysis

Each input was applied to the assembled model and the response was observed on the OLED, on the actuators and on the Windows terminal, following the test plan T01–T15 summarised in Table 3.1. The integrated system worked as intended and the subsystems did not interfere with one another.

TABLE 3.1
FUNCTIONAL TEST RESULTS

ID	Test / input	Observed response	Result
T01	Power-on	OLED initialises; firmware starts; 30 s PIR stabilisation runs	Pass
T02	Keypad entry	Digits masked; * clears; # submits	Pass
T03	Password 1234, then #	ACCESS GRANTED; main-door servo unlocks for 3 s, then relocks	Pass
T04	Wrong password, then #	ACCESS DENIED; door stays locked	Pass
T05	Authorised RFID tag	Garage opens and closes after 10 s; keypad screen not redrawn	Pass

ID	Test / input	Observed response	Result
T06	Unauthorised RFID tag	Garage does not open	Pass
T07	DHT22 reading	Temperature and humidity update about every 2 s	Pass
T08	Fan in AUTO mode	Fan follows the thresholds of Table 3.2	Pass
T09	MQ-2 reading	ADC count updates; buzzer responds at the threshold	Pass
T10	PIR + LDR	LED on for motion in darkness, within the 10 s window	Pass
T11	Bluetooth telemetry	Status report appears on the Windows terminal about every 2 s	Pass
T12	Bluetooth commands	Supported commands operate the devices and return ACK	Pass
T13	Website	Current values and device states appear via the server	Pass
T14	History	ThingSpeak history appears when configured	Pass
T15	Integration	All modules work together at the same time	Pass

3.4 Graphical / Tabular Form

The fan behaviour exercised in test T08 is given in Table 3.2 and plotted in Fig. 3.5 as regions of the temperature–humidity plane. The timing and threshold values used throughout the tests are listed in Table 3.3.

TABLE 3.2
FAN CONTROL DECISION TABLE

Condition	Fan state	PWM compare
T < 25.0 °C or RH < 80.0 %	OFF	0
T ≥ 25.0 °C and RH ≥ 80.0 %, below the next level	LOW	350
T ≥ 28.0 °C and RH ≥ 85.0 %	MEDIUM	650
T ≥ 31.0 °C and RH ≥ 90.0 %	HIGH	1000

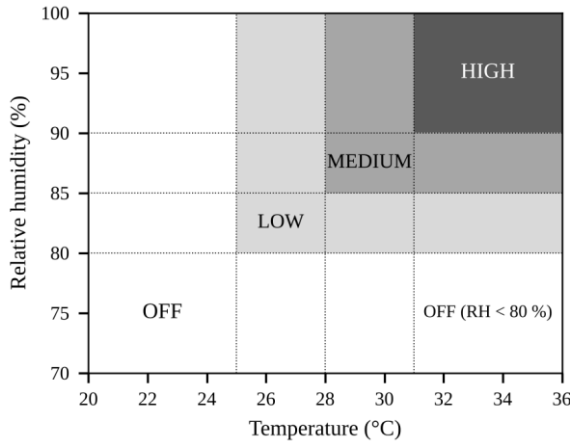


Fig. 3.5. Fan state regions in the temperature–humidity plane (configured thresholds).

TABLE 3.3
TIMING AND THRESHOLD VALUES EXERCISED IN THE TESTS

Parameter	Value
Main-door unlock time	3 s
Garage open time	10 s
PIR start-up stabilisation	30 s
PIR/LDR light window	10 s; PIR re-check at about 9 s
DHT22 interval / MQ-2 interval	about 2 s / about 500 ms
MQ-2 alert threshold	350 ADC counts (experimental)
Telemetry period	about 2 s
Bluetooth RX line buffer	80 bytes

3.5 Explanation of the Graphical / Tabulated Results

Access control. Tests T02 to T06 confirm that the two entry methods act independently. The keypad masks the digits and releases the door only for the stored password, and relocks it after 3 s without a blocking delay. The RFID path opens the garage for the authorised tag only, and because the closing is timed against the system tick, the keypad screen and the other subsystems keep running while the garage is open.

Climate and hazard. Fig. 3.5 shows that the fan is off in the large region where temperature or humidity is below its lower limit, and that the three higher states form nested regions, so a rising pair of readings passes through LOW and MEDIUM before HIGH. Because humidity must be high as well as temperature, a hot but dry room does not start the fan. The MQ-2 test shows a rising count reaching 350, at which the buzzer alert is activated; this value is a raw ADC count chosen experimentally, equivalent to about 0.28 V at the pin.

Lighting and telemetry. In test T10 the room LED behaves as an AND of motion and darkness, so the room stays unlit in daylight and when nobody moves. The 10 s window with a re-check at about 9 s keeps the light on while a person stays in the room and switches it off shortly after they leave. The Bluetooth tests T11 and T12 show that the readable status report and the acknowledged commands travel over a single serial link, from which the server and dashboard obtain the live values.

3.6 Significance of the Results Obtained

The results show that one STM32F411CEU6 can run all six subsystems cooperatively without an operating system, using timestamps rather than delays. The decision tables, thresholds and timings can be inspected and changed in one place, so the prototype is easy to demonstrate, extend and teach with. The telemetry and command set show how a local controller can be exposed to a computer without altering the local behaviour.

3.7 Deviations from Expected Results and Justification

- **MQ-2 threshold.** The alert level is a raw count, not a concentration. MQ-2 sensors need warm-up and calibration and vary between units, so 350 counts is a demonstration value and the system is not a certified smoke or gas alarm.
- **Security.** The password is fixed in the firmware and the RFID check uses only the UID. This is adequate for a demonstration but not as security.
- **DHT22 failures.** If a read fails, the last fan setting is kept; this avoids sudden fan changes but means a stale value can persist.
- **Buzzer override.** A manual BUZZER_OFF request can be overridden by the smoke alarm, deliberately, so that a hazard alert cannot be silenced remotely.

- **Telemetry format.** The firmware sends readable multi-line text, whereas the standalone WebSocket bridge supplied with the dashboard (Annexure B) expects one JSON object per line and sends JSON command envelopes. That bridge therefore needs a text parser, or the firmware output must be adapted; the PowerShell-and-Java path parses the text report directly.
- **Quantities not measured.** Power consumption, response times, sensor accuracy and Bluetooth range depend on the modules and environment and were not measured, so the report makes no claim about them.
- **Dashboard availability.** Live values depend on the COM port, bridge, server, network and ThingSpeak configuration, and the free ThingSpeak tier limits the update rate.

3.8 Conclusions

The assembled model met its target specifications in all fifteen functional tests. Access control, climate control, hazard alerting, lighting and wireless reporting operated together, and the limitations of the prototype are understood and stated. The deviations are of the kind expected from a low-cost classroom prototype and point directly to the future work in Chapter 4.

CHAPTER 4
CONCLUSION AND FUTURE SCOPE OF WORK

4.1 Work Conclusion

Summary of the work. A smart-home prototype was designed, built and tested around the STM32F411CEU6. It combines password and RFID access control, temperature- and humidity-dependent fan control, a smoke and gas alert, motion- and darkness-dependent lighting, an OLED interface and a Bluetooth telemetry and command link to a Windows computer, a Java server, ThingSpeak and a web dashboard.

Problem statement and objective. Home functions such as access, lighting, ventilation and alerts are usually handled separately. The objective was to show that one ARM microcontroller can integrate them, give local feedback and share live state with a computer.

Methodology adopted. The system was split into independent subsystems that share the main loop, the display and the serial link. Sensors and inputs are read by C firmware on the STM32 HAL, timed behaviour uses timestamps, and an HC-05, a PowerShell bridge, a Java server and ThingSpeak carry the data to the dashboard.

Conclusions. All fifteen functional tests were passed: the door, garage, fan, buzzer, room light, telemetry, commands and dashboard behaved as specified, and the subsystems worked together.

General conclusions. The STM32F411CEU6 has sufficient peripherals for a multi-sensor home controller at low cost, and a polling loop driven by timestamps is enough for a system of this size. The prototype must be regarded as a classroom demonstration, not a security- or safety-rated installation.

Significance of the results obtained. The work gives a documented pin plan, control rules, command protocol and data path that can be reused, and it records candidly which quantities remain unmeasured.

4.2 Future Scope of Work

Future work can replace the fixed password and UID-only check with stronger authentication and add authenticated, encrypted remote commands. The MQ-2 and the other sensors can be calibrated, and repeatable accuracy, response-time and power measurements can be made. The telemetry can be given robust framing, versioning and error detection, and the Windows bridge can buffer data and reconnect after a dropout. The dashboard can gain better alerts, history, export and device controls, and the hardware can move from a taped breadboard to a cleaner enclosure with terminal blocks and proper power distribution. A certified detector should be used wherever real smoke or gas safety is required, and any extra sensors should be added only after checking pin conflicts and the power budget.

REFERENCES

[1] J. Gubbi, R. Buyya, S. Marusic, and M. Palaniswami, "Internet of Things (IoT): A vision, architectural elements, and future directions," *Future Gener. Comput. Syst.*, vol. 29, no. 7, pp. 1645–1660, Sep. 2013.

[2] N. Sriskanthan, F. Tan, and A. Karande, "Bluetooth based home automation system," *Microprocess. Microsyst.*, vol. 26, no. 6, pp. 281–289, Aug. 2002.

[3] R. Piyare and M. Tazil, "Bluetooth based home automation system using cell phone," in *Proc. IEEE 15th Int. Symp. Consum. Electron. (ISCE)*, 2011, pp. 192–195.

[4] A. K. ElShafee and K. A. Hamed, "Design and implementation of a WiFi based home automation system," *World Acad. Sci., Eng. Technol.*, vol. 68, pp. 2177–2180, 2012.

[5] R. Piyare, "Internet of Things: Ubiquitous home control and monitoring system using Android based smart phone," *Int. J. Internet Things*, vol. 2, no. 1, pp. 5–11, 2013.

[6] D. Pavithra and R. Balakrishnan, "IoT based monitoring and control system for home automation," in *Proc. Global Conf. Commun. Technol. (GCCT)*, 2015, pp. 169–173.

[7] R. K. Kodali, V. Jain, S. Bose, and L. Boppana, "IoT based smart security and home automation system," in *Proc. Int. Conf. Comput., Commun. Autom. (ICCCA)*, 2016, pp. 1286–1289.

[8] STMicroelectronics, "STM32F411xC STM32F411xE datasheet," DS10314.

[9] MathWorks, "ThingSpeak documentation: Write data to a channel," ThingSpeak REST API. [Online]. Available: https://thingspeak.mathworks.com

[10] STMicroelectronics, "UM1725: Description of STM32F4 HAL and low-layer drivers," user manual.

[11] NXP Semiconductors, "UM10204: I²C-bus specification and user manual."

[12] NXP Semiconductors, “MFRC522 standard performance MIFARE and NTAG frontend,” product data sheet.

[13] ISO/IEC 14443-3, “Cards and security devices for personal identification — Contactless proximity objects — Part 3: Initialization and anticollision.”

[14] Aosong Electronics, “AM2302 (DHT22) digital temperature and humidity sensor,” data sheet.

[15] Hanwei Electronics, “Technical data: MQ-2 gas sensor,” data sheet.

[16] Guangzhou HC Information Technology Co., “HC-05 Bluetooth to serial port module,” data sheet.

[17] STMicroelectronics, “RM0383: STM32F411xC/E advanced Arm-based 32-bit MCUs,” reference manual.

ANNEXURES (OPTIONAL)

Annexure A: Bluetooth telemetry fields and commands

TABLE A.1
TELEMETRY FIELDS IN THE STATUS REPORT

Field	Values
Temperature, humidity	DHT22 readings
Smoke ADC / Smoke status	Raw count / ALERT or NORMAL
Motion	DETECTED or NONE
Light level	DARK or BRIGHT
Room light, fan mode, fan speed	Current states
Main door, garage door, garage light	Current states
DHT22 status, RFID reader status	Sensor / reader health

Annexure B: Windows serial-to-WebSocket bridge (haven_bridge.py)

The listing below is the Python bridge supplied with the dashboard. It reads one JSON object per UART line and listens only on 127.0.0.1.

```
import argparse, asyncio, json, threading, time
import serial
from serial.tools import list_ports
import websockets


clients = set()
clients_lock = threading.Lock()
loop = None
ser = None
stop_event = threading.Event()


def list_ports_now():
    ports = list(list_ports.comports())
    if not ports:
        print("No COM ports found.")
    for p in ports:
        print(f"{p.device} — {p.description}")


async def broadcast(line):
    with clients_lock:
        current = list(clients)
    if current:
        results = await asyncio.gather(*(c.send(line) for c in current), return_exceptions=True)
        with clients_lock:
            for c, r in zip(current, results):
                if isinstance(r, Exception):
                    clients.discard(c)


def serial_reader():
    buf = b""
    while not stop_event.is_set():
        try:
            chunk = ser.read(256)
            if not chunk:
                continue
            buf += chunk
            while b"\n" in buf:
                raw, buf = buf.split(b"\n", 1)
                line = raw.strip(b"\r \t")
                if not line:
                    continue
                try:
                    obj = json.loads(line.decode("utf-8", errors="replace"))
                    if not isinstance(obj, dict):
                        continue
                    text = json.dumps(obj, separators=(",", ":"))
                    print("[STM32 -> WEB]", text)
                    if loop and not loop.is_closed():
                        asyncio.run_coroutine_threadsafe(broadcast(text), loop)
                except (json.JSONDecodeError, UnicodeDecodeError):
                    print("[UART ignored non-JSON]", line[:100])
        except serial.SerialException as e:
            print("[SERIAL ERROR]", e)
            stop_event.set()
            break
        except Exception as e:
            print("[READER ERROR]", e)
            time.sleep(.2)


async def ws_handler(ws, path=None):
    with clients_lock:
        clients.add(ws)
    print("[WEB] Dashboard connected")
    try:
        await ws.send(json.dumps({"type":"bridge_status","status":"connected"}))
        async for msg in ws:
            try:
                obj = json.loads(msg)
            except json.JSONDecodeError:
                continue
            if obj.get("type") != "command":
                continue
            command = str(obj.get("command", "")).strip()
            device, sep, action = command.partition(":")
            allowed = {"main_door", "garage_door", "room_light", "garage_light", "fan", "buzzer"}
            if not sep or device not in allowed or action.upper() not in {"ON", "OFF"}:
                await ws.send(json.dumps({"type":"command_result","ok":False,"reason":"invalid_command"}))
                print("[WEB] Rejected command:", command)
                continue
            # The STM32 firmware must parse this JSON line and safely implement it.
            line = json.dumps({"type":"command","device":device,"action":action.upper()}) + "\n"
            try:
                ser.write(line.encode("utf-8"))
                ser.flush()
                print("[WEB -> STM32]", line.strip())
```

```
                await
ws.send(json.dumps({"type":"command_result","ok":True,"device":device,"action":action.upper()}))
            except serial.SerialException as e:
                print("[SERIAL WRITE ERROR]", e)
                await ws.send(json.dumps({"type":"command_result","ok":False,"reason":"serial_write_failed"}))
    except websockets.exceptions.ConnectionClosed:
        pass
    finally:
        with clients_lock:
            clients.discard(ws)
        print("[WEB] Dashboard disconnected")


async def run(args):
    global loop, ser
    loop = asyncio.get_running_loop()
    try:
        ser = serial.Serial(args.port, args.baud, bytesize=8, parity="N", stopbits=1,
                            timeout=.25, write_timeout=1)
    except serial.SerialException as e:
        print(f"Cannot open {args.port}: {e}")
        print("Check the outgoing Bluetooth COM port and close other serial monitors.")
        return 2
    print(f"[SERIAL] {args.port}, {args.baud} baud, 8-N-1")
    print(f"[WEB] ws://127.0.0.1:{args.ws_port}")
    print("Open the dashboard > Connection & settings > Connect bridge.")
    print("Bridge listens on localhost only. Press Ctrl+C to stop.")
    threading.Thread(target=serial_reader, daemon=True).start()
    try:
        async with websockets.serve(ws_handler, "127.0.0.1", args.ws_port):
            await asyncio.to_thread(stop_event.wait)
    except OSError as e:
        print("[WEBSOCKET ERROR]", e)
        return 3
    finally:
        stop_event.set()
        if ser and ser.is_open:
            ser.close()
    return 0


if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="HC-05 COM port to Haven dashboard WebSocket bridge")
    parser.add_argument("--port", help="HC-05 outgoing Bluetooth COM port, e.g. COM7")
    parser.add_argument("--baud", type=int, default=9600)
    parser.add_argument("--ws-port", type=int, default=8765)
    parser.add_argument("--list-ports", action="store_true")
    a = parser.parse_args()
    if a.list_ports:
        list_ports_now()
    elif not a.port:
        parser.error("Use --port COM7, or --list-ports")
    else:
        try:
            raise SystemExit(asyncio.run(run(a)))
        except KeyboardInterrupt:
            stop_event.set()
            print("\nBridge stopped.")
```

PROJECT DETAILS

Project title: Smart Home Automation System

Course / lab: Microcontrollers and Embedded Systems Lab (ECE 3143)

Institution: School of Electrical and Electronics Engineering, Manipal Institute of Technology, Manipal – 576104, Karnataka, India

Controller: STM32F411CEU6 (Arm Cortex-M4)

Submission: November 2026

Team details

Registration number	Roll number	Name	Responsibility
240959002	02	Nyasha Singh	Dashboard & User Interface
240959008	05	Shivansh Bose	Sensors & Environmental Safety
240959024	10	Garv Kishore Gupta	System Integration & Hardware
240959040	11	Ashwini Kodethur	STM32 Firmware & Debugging